გაკვეთილი 5 mod 3 (3.1)

🔵 ოპერატორები JavaScript-ში

ოპერატორები არის სპეციალური სიმბოლოები, რომლებიც გამოიყენება მონაცემებზე მოქმედებების შესასრულებლად.

ოპერატორები გამოიყენება:

- გამოთვლებისთვის
- შედარებისთვის
- ლოგიკური გამოთქმების შესაქმნელად
- ტექსტის დასამუშავებლად

1️⃣ არითმეტიკული ოპერატორები

არითმეტიკული ოპერატორები გამოიყენება რიცხვებზე გამოთვლების შესასრულებლად.

ოპერატორი	მნიშვნელობა
+	შეკრება
-	გამოკლება
*	გამრავლება
/	გაყოფა
%	ნაშთი

🔹 მაგალითები

```
let a = 10;
let b = 3;

console.log(a + b); // 13
console.log(a - b); // 7
console.log(a * b); // 30
console.log(a / b); // 3.333...
console.log(a % b); // 1
```

🔎 ნაშთის ოპერატორი (%)

% აბრუნებს გაყოფის ნაშთს.

მაგალითად:

```
console.log(10 % 2); // 0
console.log(7 % 2); // 1
```

გამოიყენება:

- ლუწი / კენტის გასარკვევად
- გაყოფის ნაშთის დასადგენად

2️⃣ შედარების ოპერატორები

შედარების ოპერატორები აბრუნებენ Boolean მნიშვნელობას (true ან false).

ოპერატორი	მნიშვნელობა
>	მეტია
<	ნაკლებია
>=	მეტია ან ტოლია
<=	ნაკლებია ან ტოლია
==	ტოლია
!=	არ უდრის

🔹 მაგალითები

```
let a = 5;
let b = 10;

console.log(a > b); // false
console.log(a < b); // true
console.log(a == 5); // true
console.log(a != 5); // false
```

3 ლოგიკური ოპერატორები

ლოგიკური ოპერატორები მუშაობს Boolean მნიშვნელობებზე.

ოპერატორი	მნიშვნელობა
&&	და
!	არა

◆ AND (&&)

true იქნება მხოლოდ მაშინ, თუ ორივე პირობა true-ა.

```
let a = true;
let b = false;

console.log(a && b); // false
```

◆ OR (||)

true იქნება, თუ ერთ-ერთი მაინც true-ა.

```
console.log(a || b); // true
```

◆ NOT (!)

ცვლის მნიშვნელობას საპირისპიროზე.

```
console.log(!true); // false
```

---

4 სტრიქონული ოპერატორი (+)

- გამოიყენება ტექსტის შეერთებისთვის.

```
let name = "Giorgi";
let surname = "Svanidze";

console.log(name + " " + surname);
```

მნიშვნელოვანი:

თუ ერთ-ერთი ტექსტია, + ავტომატურად ტექსტურ შეერთებას გააკეთებს.

```
console.log("რიცხვი: " + 5); // "რიცხვი: 5"
```

5️⃣ პირობითი ოპერატორი (ternary)

ეს არის მოკლე if-else ფორმა.

ფორმატი:

```
პირობა ? მნიშვნელობა1 : მნიშვნელობა2
```

პირობითი ოპერატორი არის **მოკლე ფორმა**, რომელიც გამოიყენება გადაწყვეტილების მისაღებად.

ის მუშაობს ასე:

👉 ამოწმებს ერთ პირობას
👉 თუ პირობა მართალია — აბრუნებს ერთ მნიშვნელობას
👉 თუ პირობა მცდარია — აბრუნებს მეორე მნიშვნელობას

მას ეწოდება *ternary* (სამნაწილიანი), რადგან შედგება 3 ნაწილისგან.

სტრუქტურა:

ფორმატი ასეთია:

```
პირობა ? მნიშვნელობა1 : მნიშვნელობა2
```

ანუ:

- `?` — ნიშნავს „თუ მართალია“
- `:` — ნიშნავს „სხვა შემთხვევაში“

მაგალითი

let age = 20;

let result = age >= 18 ? "სრულწლოვანია" : "არ არის სრულწლოვანი";

console.log(result);

აი რა ხდება:

1. მოწმდება პირობა → `age >= 18`
2. რადგან 20 ≥ 18 → true
3. true შემთხვევაში პროგრამა იღებს პირველ მნიშვნელობას: `"სრულწლოვანია"`

თუ age იქნებოდა 15:

1. მოწმდება პირობა → false
2. აირჩევა მეორე მნიშვნელობა: `"არ არის სრულწლოვანი"`

---

6 # ბინარული ოპერატორები (Bitwise Operators)

---

1 რა არის ბინარული ოპერატორი?

ბინარული ოპერატორები მუშაობს რიცხვის **ბიტურ წარმოდგენაზე**.

კომპიუტერი ყველა რიცხვს ინახავს 0 და 1-ების სახით.

მაგალითად:

5 = 101
3 = 011

(ეს არის მათი ბინარული ფორმა)

ბინარული ოპერატორები მოქმედებენ სწორედ ამ 0 და 1-ებზე.

---

2 რატომ ეწოდება „ბინარული“?

„Binary“ ნიშნავს ორს —
ბინარულ სისტემაში არსებობს მხოლოდ:

0
1

კომპიუტერის შიდა მუშაობა მთლიანად ამ ორ მნიშვნელობაზეა დაფუძნებული.

3️⃣ ძირითადი ბინარული ოპერატორები

ოპერატორი	სახელი
&	AND
^	XOR

4️⃣ & (AND) ოპერატორი

AND ნიშნავს:

👉 შედეგი არის 1 მხოლოდ მაშინ, როდესაც ორივე ბიტი 1-ია.

მაგალითი

მოცემულია:

```
let a = 5; // 101
let b = 3; // 011
```

ბინარულად:

```
101
& 011
```

001

შედეგი = 1

კოდში:

```
let a = 5;
let b = 3;

console.log(a & b); // 1
```

5️⃣ | (OR) ოპერატორი

OR ნიშნავს:

👉 შედეგი არის 1, თუ ერთ-ერთი მაინც 1-ია.

მაგალითი

101
| 011

111

111 = 7

```
let a = 5;
let b = 3;

console.log(a | b); // 7
```

6️⃣ ^ (XOR) ოპერატორი

XOR ნიშნავს:

👉 შედეგი არის 1, თუ ბიტები განსხვავებულია.

მაგალითი

101
^ 011

110

110 = 6

```
let a = 5;
let b = 3;

console.log(a ^ b); // 6
```

📘 პრაქტიკული სავარჯიშოები

🟢 სავარჯიშო 1 — გამოთვლა

შექმენი ორი ცვლადი და გამოიტანე:

- ჯამი
- სხვაობა
- ნამრავლი
- ნაშთი

✓ ამოხსნა

```
let a = 8;
let b = 3;

console.log(a + b);
console.log(a - b);
console.log(a * b);
console.log(a % b);
```

🟢 სავარჯიშო 2 — შედარება

მოცემულია:

```
let a = 12;
let b = 7;
```

გამოიტანე:

- a მეტია b-ზე?
- b ტოლია 7-ის?

✓ ამოხსნა

```
console.log(a > b);
console.log(b == 7);
```

🟢 სავარჯიშო 3 — ლოგიკური კომბინაცია

```
let hasID = true;
let isStudent = true;
```

გამოიტანე:

- ორივე true არის თუ არა

✓ ამოხსნა

```
console.log(hasID && isStudent);
```

🟢 სავარჯიშო 4 — ternary

მოცემულია:

```
let score = 60;
```

გამოიტანე:
„ჩაბარებულია“ ან „არ არის ჩაბარებული“

✓ ამოხსნა

```
let result = score >= 51 ? "ჩაბარებულია" : "არ არის ჩაბარებული";

console.log(result);
```

🟢 სავარჯიშო 5 — ტექსტის შეერთება

შექმენი:

- სახელი
- ასაკი

გამოიტანე ერთ წინადადებაში.

✓ ამოხსნა

```
let name = "Ana";
let age = 22;

console.log("მე მქვია " + name + " და ვარ " + age + " წლის.");
```